

Lezione 13: Classi Innestate, Classi Anonime e Lambda Expressions

Questo notebook raccoglie tutto il codice della Lezione 13 (`MavenDate`):

- `src/main/java/it/oop/core/Time.java`
- `src/main/java/it/oop/core/Date.java` (con classe statica innestata `Date.Builder`)
- `src/main/java/it/oop/core/FormattedDateConverter.java` (Interfaccia funzionale)
- `src/main/java/it/oop/core/FormattedDate.java`
- `src/main/java/it/oop/core/ItalianDate.java`
- `src/main/java/it/oop/core/AmericanDate.java`
- `src/main/java/it/oop/core/TimeStamp.java`
- `src/main/java/it/oop/ui/MainDate.java`
- `src/test/java/it/oop/core/TestItalianDate.java`

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/
├── codice-commentato/
│   └── Lezione13/
│       ├── MavenDate
│       │   ├── pom.xml
│       │   └── src
│       │       ├── main
│       │       │   ├── java
│       │       │   │   └── it
│       │       │   │       └── oop
│       │       │   │           ├── core
│       │       │   │           │   ├── AmericanDate.java
│       │       │   │           │   ├── BirthDay.java
│       │       │   │           │   ├── Date.java
│       │       │   │           │   ├── FormattedDate.java
│       │       │   │           │   ├── FormattedDateConverter.java
│       │       │   │           │   ├── ItalianDate.java
│       │       │   │           │   ├── Time.java
│       │       │   │           │   └── TimeStamp.java
│       │       │   │           └── ui
│       │       │   │               ├── Date.java
│       │       │   │               └── MainDate.java
│       │       │   └── resources
│       │       └── test
│       │           ├── java
│       │           │   ├── it
│       │           │   │   └── oop
│       │           │   │       └── core
│       │           │   │           └── TestItalianDate.java
```

Argomenti trattati:

- Pattern Builder implementato come Static Nested Class (`Date.Builder`)
- Classi anonime (`new Time() { ... }`) per implementazioni al volo di interfacce
- Interfacce funzionali SAM (Single Abstract Method) come `FormattedDateConverter`
- Espressioni Lambda sintetiche (`d -> new AmericanDate(...)`)

1. Interfaccia `Time`

```
In [1]: interface Time {
        int getSeconds();
        int getMinutes();
        int getHours();
    }
```

2. Classe `Date` con Static Nested Class `Builder`

```
In [2]: class Date {
    protected int day;
    protected int month;
    protected int year;

    public Date(int day, int month, int year) {
        this.day = day;
        this.month = month;
        this.year = year;
        verify();
    }
    public Date(int day, int month) {
        this(day, month, 2025);
    }
    public Date(Date other) {
        this.day = other.day;
        this.month = other.month;
        this.year = other.year;
        verify();
    }

    void verify() {
        if (year < 0 || month < 1 || month > 12)
            System.out.println("Illegal date!"); // Illegal date!
        else
            if (day < 1 || day > daysPerMonth(month))
                System.out.println("Illegal date!"); // Illegal date!
    }

    public int getDay() { return day; }
    public int getMonth() { return month; }
    public int getYear() { return year; }

    public static int daysPerMonth(int month) {
        int days;
        switch(month) {
            case 4:
            case 6:
            case 9:
            case 11:
                days = 30;
                break;
            case 2:
                days = 28;
                break;
            default:
                days = 31;
                break;
        }
        return days;
    }

    @Override
    public String toString() {
        return String.format("%dm%dd%d", year, month, day);
    }

    @Override
    public boolean equals(Object other) {
        if (other == null) return false;
        if (this == other) return true;
        if (!(other instanceof Date)) return false;
        Date otherAsDate = (Date) other;
        return this.day == otherAsDate.getDay() &&
            this.month == otherAsDate.getMonth() &&
            this.year == otherAsDate.getYear();
    }

    // Static Nested Class: Builder
    public static class Builder {
```

```

        private final int year;

        public Builder(int year) {
            if (year > 0)
                this.year = year;
            else
                this.year = 1970;
        }

        public Date build(int day, int month) {
            if (month < 1 || month > 12 || day < 1 || day > daysPerMonth(month))
                return new Date(1, 1, year);
            return new Date(day, month, year);
        }
    }
}

```

3. Classe Astratta `FormattedDate` e Interfaccia `FormattedDateConverter`

```

In [3]: abstract class FormattedDate extends Date {
    protected final String format;
    protected final String[] months;

    public FormattedDate(int day, int month, int year, String format, String[] months) {
        super(day, month, year);
        this.format = format;
        this.months = months;
    }

    public final String printFormat() {
        return format;
    }

    public final String getMonthAsString() {
        return months[getMonth()-1];
    }

    public abstract String prettyPrint();
}

@FunctionalInterface
interface FormattedDateConverter {
    FormattedDate convert(FormattedDate date);
}

```

4. Classi `ItalianDate` e `AmericanDate`

```

In [4]: class ItalianDate extends FormattedDate {
    private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",
    "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };

    public ItalianDate(int day, int month, int year) {
        super(day, month, year, "dd/mm/yyyy", MONTHS_IT);
    }

    @Override
    public String prettyPrint() {
        return day + " " + getMonthAsString() + " " + getYear();
    }

    @Override
    public String toString() {
        return day + "/" + getMonth() + "/" + getYear();
    }
}

class AmericanDate extends FormattedDate {
    private static final String[] MONTHS_US = { "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December" };

    public AmericanDate(int day, int month, int year) {

```

```

        super(day, month, year, "mm/dd/yyyy", MONTHS_US);
    }

    @Override
    public String prettyPrint() {
        return getMonthAsString() + " " + day + ", " + getYear();
    }

    @Override
    public String toString() {
        return getMonth() + "/" + getDay() + "/" + getYear();
    }
}

```

5. Classe `MainDate` ed Esecuzione

```

In [5]: class MainDate {
    public static void main(String[] args) {
        Date.Builder dateBuilder = new Date.Builder(2025);
        Date d1 = dateBuilder.build(10, 9);
        Date d2 = dateBuilder.build(10, -1);
        System.out.println(d1.toString()); // y2025m9d10
        System.out.println(d2.toString()); // y2025m1d1

        // Classe anonima che implementa l'interfaccia Time
        Time init = new Time() {
            @Override
            public int getHours() { return 0; }
            @Override
            public int getMinutes() { return 0; }
            @Override
            public int getSeconds() { return 0; }
            @Override
            public String toString() {
                return String.format("%02d:%02d:%02d", getHours(), getMinutes(), getSeconds());
            }
        };
        System.out.println(init.toString()); // 00:00:00

        // Espressione Lambda che implementa FormattedDateConverter
        FormattedDateConverter toAmerican =
            d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear());
        System.out.println(toAmerican.convert(new ItalianDate(11, 11, 2025)) instanceof AmericanDate); //
true
    }
}
MainDate.main(null);

```

```

y2025m9d10
y2025m1d1
00:00:00
true

```